

Tutorial: Implement the missing **Snake** game functions (Advanced)

Use the starter file and implement the methods so they meet the behavior rules and satisfy the **IGame** interface.

Repo-specific architecture: claim code is injected

In this repo, the 6-digit claim code is **not generated inside the game**.

Flow:

1. Game sets `gameOver = true` and exposes it via `IsGameOver()`.
2. `ConsoleTest/Program.cs` detects game over, mints a claim code (server-backed), then calls `SetGameOverCode(claimCode)`.
3. Program displays the claim code on the emulator/hardware display.

Therefore:

- `GetGameOverCode()` may return null until the program injects a value.
- The game must **not overwrite** an injected code.

Required **IGame** members (must be implemented)

- `void Initialize(IPixel[,] pixels)`
- `void Update(IPixel[,] pixels)`
- `void DrawTitle(IPixel[,] pixels)`
- `void HandleInput(ConsoleKey key, ref bool stateChanged)`
- `int GetScore()`
- `bool IsGameOver()`
- `string? GetGameOverCode()`
- `void SetGameOverCode(string? code)`
- `string GameId { get; }`

Recommended state fields (example)

```
private readonly Queue<(int x, int y)> snake = new Queue<(int x, int y)>();
private int snakeLength = 5;
private int headX = 10;
private int headY = 5;

private int directionX = 0;
private int directionY = 1;
private int nextDirectionX = 0;
private int nextDirectionY = 1;
```

```
private readonly Random rand = new Random();
private (int x, int y) food;

private int score = 0;
private float rainbowShift = 0f;

private bool gameOver = false;
private bool wallsAreDeadly = false;
private int moveCounter = 0;

// Injected claim code (nullable until Program sets it)
private string? gameOverCode;

// IGame requirement (used by Program when minting claim codes)
public string GameId { get; } = "REPLACE_WITH_REAL_GAME_ID";
```

Core behavior requirements

Initialize(IPixel[,] pixels)

- Reset all gameplay state:
 - snake body, length
 - head position
 - direction + buffered direction
 - score, counters, wall mode, `gameOver`
- Reset the injected claim code:
 - `gameOverCode = null`
- Spawn food within bounds in a valid cell
- Build a starting snake of multiple segments

Update(IPixel[,] pixels)

- If `gameOver`:
 - render game-over visuals
 - do not advance gameplay state
- Timed movement:
 - gate movement with a speed rule (movement interval decreases as score increases)
 - must remain playable (never ≤ 0)
- Apply buffered direction only at the movement step
- Move the head by one cell each movement step
- Walls:
 - wrap-around when deadly walls are disabled
 - end game when deadly walls are enabled and head exits bounds
- Self-collision ends the game
- Food collision:
 - increment score
 - increment snake length

- respawn food in an empty cell
- Update the snake queue correctly so growth works
- Draw the current frame each update call

7-seg/claim-code requirement (repo-specific):

- On death, set `gameOver = true` and render game over.
- Do not generate a claim code in-game.
- Do not clear/overwrite `gameOverCode` except during `Initialize`.

HandleInput(ConsoleKey key, ref bool stateChanged)

- Read direction input (WASD + arrows)
- Buffer direction changes via `nextDirectionX/nextDirectionY`
- Prevent illegal instant reversal (block opposite of current direction)
- Escape requests leaving play state via `stateChanged = true`
- When `gameOver`:
 - allow Escape to return to title
 - otherwise ignore movement inputs

Scoring & state access

- `GetScore()` returns current score
- `IsGameOver()` reports ended state
- `GetGameOverCode()` returns the injected claim code (nullable)
- `SetGameOverCode(code)` stores the injected claim code

Rendering

- `DrawTitle()` must be visually distinct from gameplay
- `DrawGame()` renders background + snake + food
 - head must be visually distinct
- `DrawGameOver()` renders an unmistakable game-over state distinct from gameplay

Acceptance checklist (quick)

- All `IGame` methods compile and behave correctly
- No in-game call to `CodeGenerator` for claim code
- `gameOverCode` is nullable and only set via `SetGameOverCode` (except reset in `Initialize`)
- Program can:
 - show score during play (`GetScore`)
 - transition when dead (`IsGameOver`)
 - display server code after death (via `SetGameOverCode` / `GetGameOverCode`)